

Documentación del Proyecto BidFinder

Marcos Gomá 875253
Pablo Villa 874773
Javier Jimenez 869756
Pablo Latre 778043

URLs de Acceso y Swagger

Frontend (Aplicación Web): <https://bid-finder-frontend-web.vercel.app/>

Backend (API): <https://bid-finder-backend.onrender.com/api/v1>

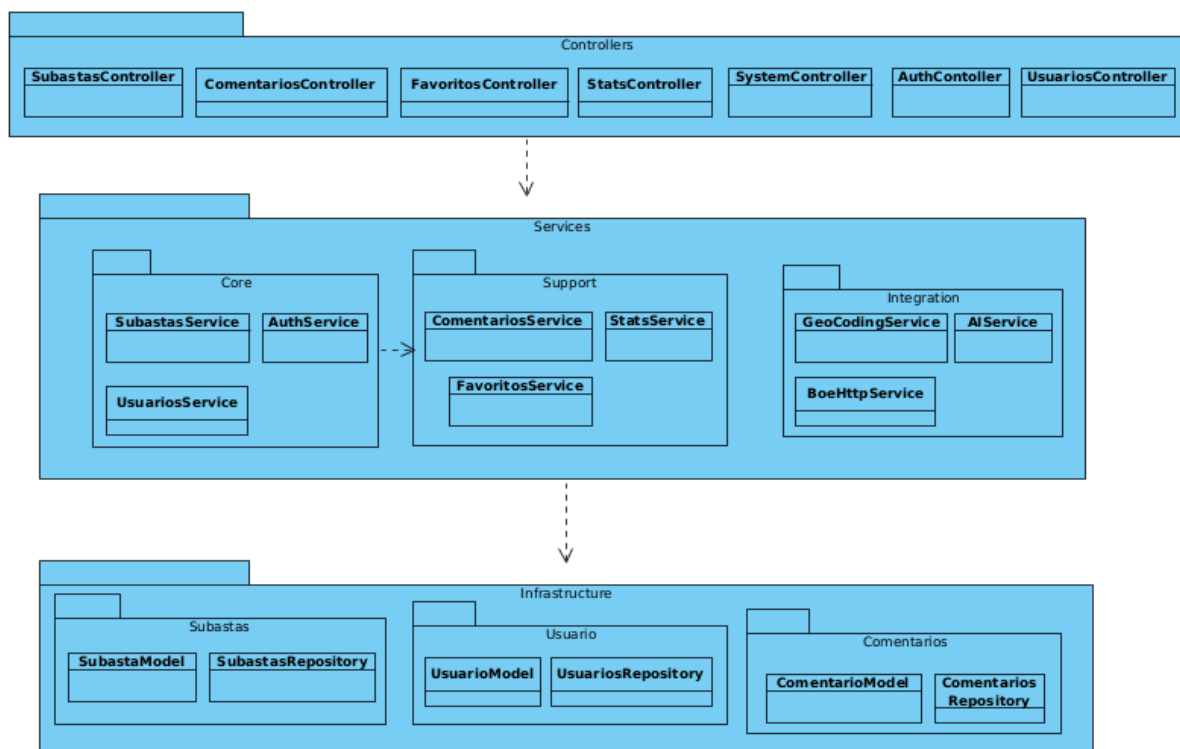
Documentación Swagger: <https://bid-finder-backend.onrender.com/api-docs>

Credenciales de Acceso

Para evaluar la aplicación sin necesidad de registrarse, se han creado estas cuentas de prueba:

- Usuario normal:
 - Email: demo@bfinder.com
 - Contraseña: 123456
- Administrador:
 - Email: admin@bfinder.com
 - Contraseña: admin123

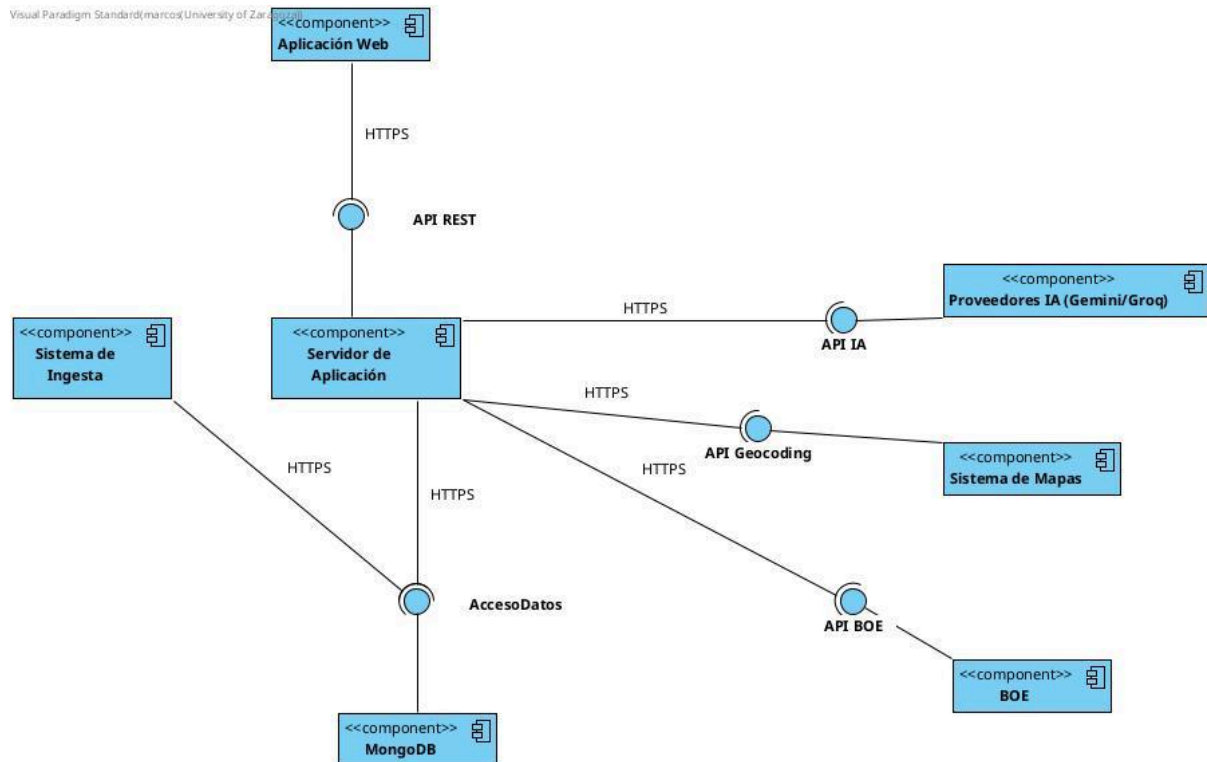
Arquitectura de Componentes



Controlador	Responsabilidad
SubastasController	Obtención de subastas, general y específica.
ComentariosController	Obtención de comentarios, general y específico, creación y eliminación.
FavoritosController	Creación, eliminación y obtención de comentarios de un usuario.
StatsController	Obtención de subastas por provincia y categoría.
SystemController	Obtener estado del sistema (health)
AuthController	Registro, login y refresh token.
UsuariosController	Obtención de usuarios para control de los mismos.

Servicio	Subpaquete	Responsabilidad
SubastasService	Core	Lógica de negocio principal sobre subastas: aplicar filtros de búsqueda, paginación, ordenación, gestionar el estado de procesado.
AuthService	Core	Registro de usuarios, validación de credenciales, generación y verificación de tokens JWT, gestión de refresh tokens.
UsuariosService	Core	Mostrar a los usuarios para control de los mismos (admin).
ComentariosService	Support	Gestión de comentarios (creación, eliminación y obtención).
FavoritosService	Support	Gestión de la relación usuario-subasta en favoritos.
StatsService	Support	Cálculo de estadísticas: número de subastas, porcentaje de subastas procesadas...
GeoCodingService	Integration	Abstracción para comunicarse con la API de geolocalización (Nominatim).
AIService	Integration	Abstracción para comunicarse con los proveedores de IA (Gemini, Groq). Incluye la lógica de selección de proveedor, validación de respuestas y actualización de subastas.
BoeHttpService	Integration	Abstracción para comunicarse con el BOE.

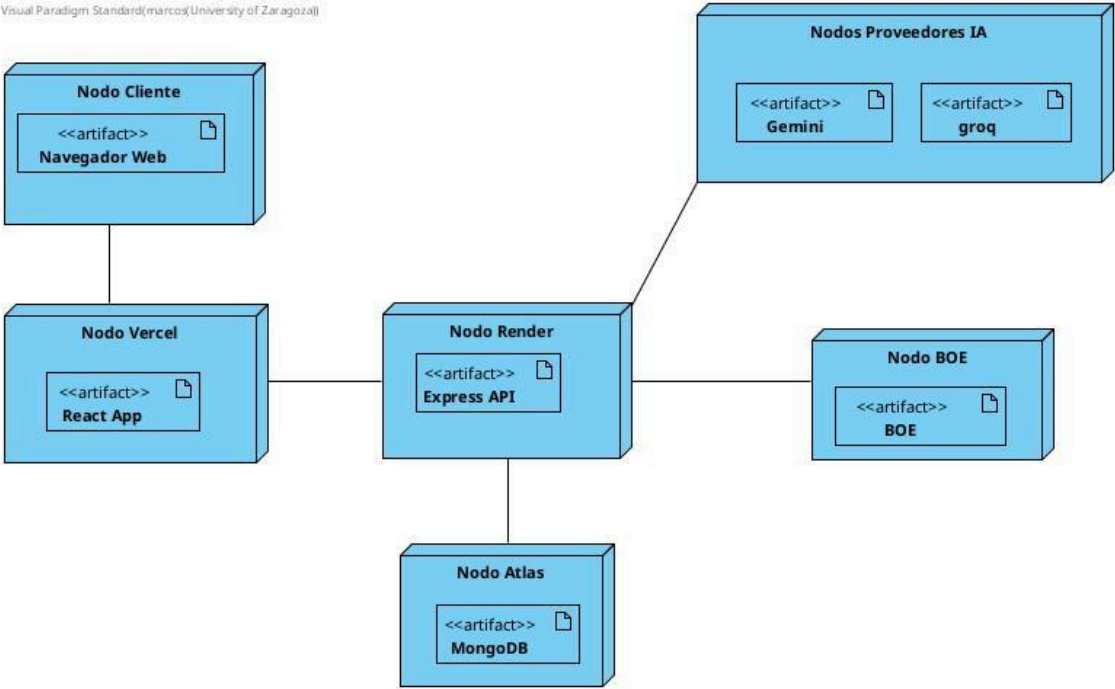
Infraestructura	Modelo	Repositorio
Subastas	SubastasModel	SubastasRepository
Usuarios	UsuariosModel	UsuariosRepository
Comentarios	ComentariosModel	ComentariosRepository



Componente	Tipo	Descripción
Sistema de Ingesta	<<component>>	Módulo encargado del proceso ETL: descarga el sumario del BOE, filtra anuncios de subastas, descarga los XML y almacena el texto bruto en la base de datos.
Aplicación Web	<<component>>	Cliente SPA desarrollado en React + TypeScript. Se ejecuta en el navegador del usuario.
Servidor de Aplicación	<<component>>	Backend central (Node.js + Express). Orquesta toda la lógica de negocio: autenticación, gestión de subastas, comentarios, roles de administrador.
MongoDB	<<component>>	Base de datos NoSQL (desplegada en MongoDB Atlas). Almacena toda la información persistente del sistema.
Proveedores IA	<<component>>	Abstracción de los servicios de IA, concretamente Gemini y Groq, pero se pueden añadir más
Sistema de Mapas	<<component>>	Servicio externo de geolocalización (Nominatim). Convierte direcciones en coordenadas (latitud, longitud).
BOE	<<component>>	Sistema externo del Boletín Oficial del Estado. Proporciona los datos abiertos a través de su API REST.

Conector	Protocolo	Formato Datos	Desde	Hacia
Acceso Datos	HTTPS	BSON (Binario de Mongo)	Servidor Aplicación	MongoDB
			Sistema de Ingesta	
API REST	HTTPS	JSON	Aplicación Web	Servidor Aplicación
API IA	HTTPS	JSON (prompt + respuesta estructurada)	Sistema de Ingesta	Proveedores IA
API Geocoding	HTTPS	JSON (parametrizado por dirección)	Servidor Aplicación	Sistema de Mapas
API BOE	HTTPS	JSON (resumen) y XML (anuncios)	Servidor Aplicación	BOE

Visual Paradigm Standard(marcos(University of Zaragoza))



Nodos	Tipo Implícito	Artefacto desplegado	Descripción
Nodo Cliente	<<node>> (dispositivo usuario)	Navegador Web	Representa el equipo del usuario final (PC, móvil, tablet) donde se ejecuta el navegador. Es el punto de acceso a la aplicación.
Nodo Vercel	<<node>> (plataforma cloud PaaS)	React App	Nodo que aloja el frontend de la aplicación. Vercel sirve los archivos estáticos de la SPA construida con React + TypeScript.
Nodo Render	<<node>> (plataforma	Express API	Nodo donde se ejecuta el backend (Node.js + Express). Render aloja la lógica de

	cloud PaaS)		negocio, controladores, servicios y el worker de IA.
Nodo Atlas	<<node>> (MongoDB Atlas cloud)	MongoDB	Nodo que proporciona la base de datos NoSQL gestionada (MongoDB Atlas). Almacena todas las colecciones (usuarios, subastas, comentarios, etc.).
Nodo BOE	<<node>> (plataforma cloud PaaS)	BOE	Servidor del Boletín Oficial del Estado. Proporciona la API de datos abiertos (sumarios y XML).
Nodo Proveedores IA	<<node>> (plataforma cloud PaaS)	Gemini	Plataformas de IA externas (Google Gemini, Groq). El nodo agrupa a ambos proveedores.

Fuentes de Datos Abiertos

En este proyecto se ha utilizado el portal de datos abiertos del Boletín Oficial del Estado (BOE). Se ha implementado un proceso ETL automatizado por un cron job. Un controlador de ingesta descarga el BOE del día, filtra los anuncios correspondientes a la sección de subastas, descarga el XML de cada anuncio y almacena el texto bruto en la base de datos para su posterior procesamiento por la IA.

Cabe destacar que, en el despliegue final, la plataforma donde hemos desplegado nos cobraba por poner cron jobs. Por esto, hemos desarrollado una tarea de seeding que ejecuta el proceso ETL para X días anteriores. De esta forma podemos poblar la base de datos para la demostración sin tener que pagar.

URLs utilizadas: <https://www.boe.es/datosabiertos/api/boe/sumario/%7Bfecha%7D> y descarga directa de los XMLs de los anuncios de subastas.

Módulos del Backend

El backend se ha implementado utilizando *Node.js* como entorno de ejecución. Para su desarrollo se han usado varias librerías especializadas, cada una de una función concreta. Esta organización modular permite mantener el código de forma más sencilla y utilizar únicamente las dependencias necesarias.

El servidor se ha implementado con *Express*, un framework para *Node.js* que permite definir los endpoints de la API, gestionar las peticiones HTTP y organizar la lógica. A partir de esta estructura se implementan funcionalidades como la autenticación de usuarios, la validación de los datos recibidos en las peticiones y la gestión de errores de forma centralizada.

Para la interacción con la base de datos se ha utilizado *Mongoose*, un ODM (Object Data Mapper) para *MongoDB*. Esta librería permite definir esquemas para las distintas

colecciones de la aplicación como usuarios, subastas y comentarios, así como validar los datos y realizar consultas complejas.

La autenticación de usuarios se basa en JSON Web Tokens (JWT), implementados mediante la librería *jsonwebtoken*. Este sistema permite mantener sesiones sin almacenar información en el servidor, generando tokens que identifican al usuario.

Las contraseñas de los usuarios se almacenan cifradas utilizando *bcryptjs*, que aplica funciones hash con salt para evitar que puedan recuperarse en caso de acceso no autorizado a la base de datos.

Continuando con temas de seguridad, también se ha usado la librería *cors* para controlar qué dominios pueden hacer peticiones a la API, limitando el acceso al frontend.

En el proceso de ingesta de información procedente del BOE, se utiliza *fast-xml-parser*, que convierte los archivos XML descargados en objetos JavaScript que pueden ser procesados por la aplicación.

Para la integración con modelos de inteligencia artificial se ha utilizado el SDK oficial *@google/generative-ai*, con el cual se accede a Gemini para sacar información estructurada a partir de los textos legales.

Se ha utilizado *axios* como cliente HTTP para realizar peticiones a servicios externos como la descarga del BOE, consultas a APIS de geolocalización y llamadas a modelos de lenguaje.

El registro de actividad se hace con *Morgan* y *Winston*. *Morgan* (logger por defecto en Express) se encarga de registrar las peticiones HTTP recibidas, mientras que *Winston* permite almacenar mensajes internos clasificados por nivel de severidad, lo cual es muy útil para la depuración y seguimiento en producción. Cabe destacar que se utiliza *Morgan* ya que es la manera estándar de registrar peticiones HTTP en *Express*, sin embargo, estos registros se puentean luego con *Winston* para que se guarden en su registro.

La documentación de la API se ha generado con *swagger-jsdoc* y *swagger-ui-express*, que permiten crear automáticamente una interfaz interactiva basada en el estándar OpenAPI a partir de comentarios en el código.

Para las pruebas se han utilizado *Jest* y *Supertest*. *Jest* permite ejecutar pruebas unitarias mientras que *Supertest* facilita la comprobación de los endpoints con peticiones HTTP simuladas.

Para las pruebas de integración se ha utilizado *mongodb-memory-server*, que levanta una instancia temporal de *MongoDB* en memoria. Esto nos permite ejecutar los tests en un entorno aislado sin afectar a la base de datos de desarrollo o producción.

Prototipado de la Solución

El diseño previo de la interfaz de usuario se realizó en Figma: [Link Prototipado Figma](#)

Tecnología y Módulos del Frontend

La parte del cliente de BidFinder se ha desarrollado como una *Single Page Application* (SPA). Esto permite que la navegación entre las distintas secciones de la app se haga sin recargar la página y de forma más fluida para el usuario.

Para su implementación se ha utilizado *React 19* como librería principal para la construcción de la interfaz de usuario, con *TypeScript* que aporta tipado estático y facilita la detección de errores durante el desarrollo.

Como herramienta de desarrollo y empaquetado se ha utilizado *Vite*, que ofrece un entorno de trabajo rápido, reduciendo los tiempos de compilación y mejorando la experiencia durante el desarrollo.

Para la gestión de rutas se ha utilizado *react-router-dom*, que permite definir las distintas vistas de la aplicación y navegar entre ellas sin recargar la página. Además, esta librería hace posible proteger ciertas rutas para restringir el acceso a funcionalidades, como el panel de administración.

El diseño de la aplicación se ha realizado con *Tailwind CSS*, un framework que permite desarrollar interfaces responsive de forma fácil y ordenada. Además, fue bastante sencillo utilizarlo gracias a que ya lo habíamos visto en clase y sabíamos como funcionaba. Para los iconos se han usado las librerías *lucide-react* y *@phosphoricons/react*. Estas nos proporcionan iconos consistentes y que se pueden personalizar de forma sencilla. Las notificaciones al usuario se han gestionado principalmente con *react-hot-toast*, que muestra mensajes informativos tras acciones o muestran errores.

Una de las funcionalidades principales de la aplicación es mostrar las subastas en un mapa interactivo. Para esto, se han utilizado *Leaflet* y su integración con React con *react-leaflet*. Como varias subastas se pueden encontrar en ubicaciones cercanas, se añadió *react-leaflet-cluster*, que agrupa automáticamente marcadores cercanos y los desglosa a medida que se hace zoom en el mapa. Esto ayuda en gran medida a mejorar la claridad del mapa y evitar la saturación visual.

En el panel de administración se han utilizado *Chart.js* y *react-chartjs-2* para generar gráficos diversos que permiten al administrador analizar información.

Para las pruebas unitarias se ha utilizado *Vitest*, junto con *@testing-library/react*, que permite verificar el comportamiento de los componentes simulando la interacción del usuario. Las pruebas end-to-end se han realizado con *Cypress*, reproduciendo flujos completos en un navegador real.

Por último, se han utilizado *ESLint* y *Prettier* para mantener un estilo de código uniforme y libre de bugs.

Integración de Inteligencia Artificial

El principal reto del proyecto era que los anuncios de subastas publicados en el BOE son textos largos, poco estructurados y que no siguen un patrón claro. Es por eso que era prácticamente imposible extraer información clave de ellos con expresiones regulares, además, tampoco le podemos presentar ese texto al usuario ya que es complicado de leer. Por esto, decidimos introducir la IA en el proyecto como un motor de extracción y estructuración de datos. El trabajo de la IA es leer cada anuncio, comprenderlo, y devolvernos la información que ha encontrado de forma estructurada.

Para implementar el módulo de IA, hemos seguido los principios de arquitectura del software SOLID, abstrayendo el código lo máximo posible de modelos concretos. De esta forma, contamos con un servicio de IA que no tiene ni idea de que modelos utiliza la app, simplemente se le inyectan (inversión de dependencias) una serie de providers donde se encuentra la lógica necesaria para mandar peticiones a modelos específicos. De esta forma, el módulo está abierto a la extensión ya que se le pueden añadir todos los providers que se necesite y cerrado a las modificaciones (open / closed). Este mecanismo resulta increíblemente útil ya que al poder añadir los providers que queramos, nos aseguramos de que si un modelo se queda sin tokens para devolver una respuesta, el sistema salta automáticamente al siguiente. En nuestro caso, hemos comprobado que con dos providers es suficiente para procesar las subastas que necesitábamos para poblar la base de datos.

Concretamente, los dos providers que hemos seleccionado han sido *Gemini 2.5 Flash* de Google y *Groq* ejecutando *Llama 3* de Meta. El primer modelo que se ejecuta por defecto es *Gemini*, elegimos este por varios motivos. En primer lugar, lo hicimos para aprovechar la suscripción pro que nos proporciona la universidad. Gracias a esto, la ventana de contexto que tiene es muy larga y en varias pruebas hemos comprobado que con este modelo obtenemos resultados muy sólidos. A pesar de todo esto, la API sigue teniendo un límite diario, por lo que tuvimos que seleccionar otro modelo de fallback. Para esto se eligió *Groq* ya que tiene una API que permite a los desarrolladores ejecutar modelos de código abierto de forma gratuita. También cuenta con un límite, pero como ya hemos dicho, con la combinación de estos dos modelos nos ha sido suficiente para el proyecto.

Para comunicarnos con el modelo utilizamos un prompt muy restrictivo. Le asignamos el rol de un experto legal y analista de datos, y le prohibimos terminantemente que nos salude, se despida o utilice formato markdown ya que solo queremos que devuelva un objeto JSON. Le pasamos una estructura de claves que debe rellenar, pidiéndole que extraiga cosas como un título resumido, precio de salida, valor de tasación, dirección o un resumen del texto. Además, le pedimos que comprenda el texto y a partir de este deduzca cosas como si hay ocupantes en la vivienda o si tiene cargas previas (hipotecas, usufructos). Basándose en eso clasifica el riesgo de la inversión el Alto, Medio o Bajo. Como ya hemos dicho, estos anuncios no tienen ninguna estructura fija, por lo que en el prompt especificamos que si no

encuentra algún dato, debe devolver en ese campo nulo. Así evitamos que alucine e invente información.

La integración con el modelo de datos de la aplicación se hace de forma asíncrona para evitar bloquear el servidor. Una vez que se descargan y almacenan los textos del BOE, se inicia un worker de IA. Este proceso coge las subastas de la base de datos con estado “PENDIENTE” y las va procesando con la IA. Cuando se recibe el JSON de respuesta, no confiamos ciegamente ya que pasamos estos datos por un validador que comprueba los tipos de datos y filtra alucinaciones comunes. En este momento, si la ubicación extraída es válida, la mandamos al servicio de mapas (*Nominatim*) para obtener las coordenadas exactas. Al terminar todo esto, se actualiza la subasta en la base de datos y su estado pasa a ser “PROCESADO”.

Validación y Pruebas

La estrategia de validación del proyecto se ha aplicado en todo el stack tecnológico, abarcando desde el tipado estático hasta pruebas end-to-end. A continuación, se detalla el enfoque adoptado tanto para el frontend como para el backend.

En la capa del frontend, la validación sigue tres enfoques distintos. En primer lugar tenemos el tipado estático, facilitado por el uso de TypeScript, mediante la definición de interfaces estrictas que actúan como contrato de datos. Por otra parte, se ha implementado una suite de 55 pruebas automatizadas utilizando *Vitest* y *React Testing Library*. El objetivo de esta suite es validar el correcto renderizado de componentes clave y asegurar que los hooks y servicios de conexión a la API funcionan correctamente. Por último se han desarrollado 2 flujos end-to-end que garantizan el correcto funcionamiento de aspectos críticos de la aplicación como son la navegación hacia el panel interactivo del mapa y la exploración detallada de una subasta y su guardado en favoritos.

En el backend es donde encontramos una suite de validación más importante. Se ha utilizado el framework *Jest* para orquestar 237 tests divididos en 41 suites. Dentro de esta batería de tests hemos implementado tanto tests unitarios como de integración. En primer lugar, las pruebas unitarias se encargan de testear de forma aislada controladores, servicios y utilidades entre otros. En estos se hace uso de mocks y stubs para simular las respuestas de los repositorios o de APIs externas, permitiendo validar la lógica pura de negocio, el control de roles y el manejo de excepciones de forma muy rápida. Por otra parte tenemos pruebas de integración, las cuales validan principalmente el acceso a datos. Estas pruebas son las que comprueban los repositorios y, en ellas se ha utilizado la herramienta *mongodb-memory-server*. Esta herramienta levanta una instancia *MongoDB* en memoria durante la ejecución de los tests. Esto nos permite comprobar funciones complejas como la paginación o agregaciones para contar totales.

Desarrollos Opcionales

Para asegurar la calidad, mantenibilidad y escalabilidad del proyecto, se han implementado varios de los desarrollos opcionales propuestos. Aunque la puntuación máxima alcanzable en este apartado es de 1,5 puntos, el proyecto cubre hasta 2,5 puntos ya que algunas de ellas van de la mano y son realmente útiles desde el inicio del proyecto:

1. Utilizar herramientas de integración continua en el desarrollo:

Desde las primeras iteraciones del proyecto se implementó un flujo de integración y despliegue continuo utilizando *Github Actions*. Se separaron los flujos del frontend y del backend para mantener la independencia de los repositorios.

En ambos flujos, primero se implementó la parte de integración continua, que es la que más se ha ido utilizando durante el desarrollo ya que se dispara automáticamente con cada push o pull request en las ramas main y dev. En esta parte, tanto en backend como en frontend se ejecuta el linter y las suites completas de tests unitarios. Luego, en el backend se realiza un test de construcción de la imagen de *Docker* para asegurar que el contenedor se puede montar sin errores y en el frontend se comprueba que el código compila correctamente. Esto ha resultado extremadamente útil ya que nos obliga a mantener un código de calidad durante todo el desarrollo, además, al utilizar *Github Actions*, en caso de que el flujo falle llega un correo avisando de esto.

En la segunda mitad del proyecto, cuando ya se vieron los temas de despliegue en clase, se añadió la parte de entrega continua al flujo. Optamos por desplegar el backend de nuestra aplicación en *Render* y el frontend en *Vercel*. Aunque ambas plataformas tienen funciones que monitorizan tu repositorio y cuando subes cambios actualizan el despliegue, decidimos integrar esto en nuestros flujos de *Github Actions* para enriquecerlos y aprender. Concretamente, configuramos webhooks para lanzar el despliegue automático sólo si la integración continua pasa en verde y los cambios van a la rama main.

Por último, para cumplir con otro de los desarrollos opcionales (75% de cobertura demostrable en backend), en ese flujo concreto se añadió tras el testing con cobertura una tarea para subir los artefactos resultantes (el informe de cobertura) a *Github*.

2. Alcanzar un 75% de cobertura de código demostrable en el backend:

El backend ha sido testeado utilizando el framework de *Jest*. Alcanzar una buena cobertura no ha resultado difícil gracias a la arquitectura del proyecto, basada en la inversión de dependencias y una estricta separación por capas (rutas, controladores, servicios y repositorios). Esta modularidad nos ha permitido aislar cada componente, facilitando la creación de mocks para simular la base de datos o las respuestas de las APIs externas sin necesidad de levantar toda la infraestructura.

A pesar de que lo exigido era un 75% de cobertura, hemos superado esta cifra debido a que el testing estaba ya integrado en nuestro proceso de desarrollo. Se han redactado un total de 237 tests distribuidos en 41 suites, logrando las siguientes métricas globales:

- Cobertura de Sentencias (Statements): 91.95%

- Cobertura de Líneas (Lines): 93.16%
- Cobertura de Funciones (Functions): 90.97%
- Cobertura de Ramas lógicas (Branches): 81.50%

Finalmente, para cumplir con el requisito de que esta cobertura sea completamente demostrable, hemos integrado la generación de reportes en nuestro pipeline. Cada vez que se ejecutan las pruebas en *GitHub Actions*, el sistema compila un informe HTML navegable y lo sube como un artefacto descargable (backend-coverage-report.zip). De esta forma, existe un registro transparente de la cobertura del código tras cada commit.

3. Utilizar herramientas de integración continua en el desarrollo:

Como ya se ha comentado antes, una de las cosas que decidimos al inicio del proyecto fue configurar un linter que analizara nuestro código e integrarlo en el flujo CI/CD. Se configuró *ESLint* como analizador estático, el cual garantiza legibilidad, prevención de bugs y unificación del estilo de programación entre todos los miembros del equipo. Se utiliza tanto en el repositorio del frontend (integrado con TypeScript) como en el del backend.

En cuanto a los detalles de configuración, decidimos utilizar una configuración estricta donde cualquier warning se trata como un error bloqueante. Esto nos ha permitido acumular muy poca deuda técnica a lo largo de los sprints y ha facilitado tanto añadir nuevas funcionalidades como refactorizar partes del código ya existentes.

Mejoras Implementadas

A lo largo del desarrollo y en parte gracias al testing (al tratar de meter casos de prueba que no funcionarían porque nuestra implementación no soportaba esos escenarios), fuimos introduciendo mejoras. Estas no estaban previstas inicialmente en los requisitos planteados, sin embargo, aportan robustez y profesionalidad a la aplicación.

Una de las principales mejoras implementadas fue el ya comentado sistema de providers de la IA. Inicialmente, nuestro sistema estaba muy acoplado con el modelo de *Gemini*, y, pronto comprobamos que con un solo modelo no era suficiente para procesar todas las subastas. Esta mejora puede que sea la más importante ya que nos ahorró mucho tiempo (sino hubiéramos tenido que sacar mas API Keys, lo cual suele ser tedioso) y nos permitió poblar las BDs de desarrollo.

Otro problema que encontramos es que las direcciones que vienen en los anuncios de las subastas pueden ser bastante extrañas o incluir textos como por ejemplo “parcela M6-DI”. Esto confundía a los motores de búsqueda de localizaciones y no eran capaces de darnos una respuesta. Para solucionar esto, realizamos geolocalización en cascada. Si el sistema no encuentra la dirección exacta, hace otro intento limpiando la dirección de ruido. Si esto también falla, busca únicamente el centro del municipio para garantizar que la subasta siempre aparezca en el mapa.

Por otra parte, también realizamos mejoras en el sistema de autenticación. Inicialmente implementamos únicamente autenticación con un token de acceso JWT con una duración alta, ya que si ponemos una duración más baja y segura arruinamos la experiencia de

usuario. Sin embargo, tras pensarlo mejor, decidimos implementar un sistema de Refresh Tokens. El token principal dura muy poco, pero el sistema utiliza un segundo token de larga duración (almacenado de forma segura en la BD) para ir renovando la sesión sin que el usuario se entere.

Además de esto, también con el objetivo de mejorar la experiencia del usuario, decidimos mejorar nuestro buscador de subastas. En principio funcionaba solamente por palabras, sin embargo, para que fuera útil de verdad añadimos un motor de búsqueda con sistema de pesos. Para esto, principalmente tuvimos que modificar el modelo de la base de datos, indicando qué campos cuentan con mayores pesos. Así, si un usuario busca “piso en Madrid”, el sistema le da más importancia si las palabras aparecen en el título o en la zona a que si aparecen en el texto bruto. Actualmente es difícil comprobar la efectividad de esto ya que, el mapa tiene un límite de subastas que pueden aparecer pero, al no haber demasiadas en la BD, al final se van a mostrar todas. Sin embargo, pensando en producción, cuando haya miles de subastas, esto es clave para que el usuario encuentre lo que necesita.

La última mejora añadida fue respecto a los requisitos que planteamos. Al comenzar el proyecto, decidimos crear a parte del script de ingesta diaria que ejecutaría el cron, un script de seeding que se ejecuta para los X días anteriores. Esto lo hicimos con el objetivo de poblar nuestras BDs locales y poder realizar pruebas con datos. Sin embargo, esto se volvió clave cuando nos dimos cuenta que la plataforma donde desplegamos el backend (*Render*), cobra por configurar cron jobs en sus servidores. Como no queríamos pagar para esto, lo que hicimos finalmente fue aprovechar este script para poblar la BD de producción.

Valoración Global del Proyecto

La valoración global del proyecto BidFinder es bastante positiva, ya que se ha conseguido montar una aplicación funcional que cumple el objetivo principal que nos marcamos, coger el caos de datos y textos que publica el BOE, procesarlos mediante Inteligencia Artificial y mostrarlos de forma estructurada en un mapa interactivo. Ha sido un reto conectar tantas piezas, desde el proceso ETL de extracción diaria hasta la geolocalización de las direcciones, pero el resultado final demuestra que la idea era viable y aporta valor real.

A nivel técnico, estamos muy satisfechos con la base que hemos construido. Hemos aplicado con éxito principios de Clean Architecture y metodologías ágiles, lo cual nos ha permitido coordinar el trabajo en paralelo entre el frontend y el backend sin problemas. Estamos especialmente orgullosos del nivel de madurez del proyecto al contar con un pipeline de Integración y Despliegue Continuo automatizado y con el alto nivel de cobertura de tests logrado.

La decisión de integrar Inteligencia Artificial como el motor principal del proceso de ingesta ha sido un acierto. Esto nos ha obligado a investigar el ecosistema actual de LLMs, comprender la importancia de la validación de los datos y diseñar sistemas tolerantes a fallos, como la arquitectura de providers implementada para garantizar la alta disponibilidad del servicio.

En conclusión, BidFinder ha cumplido satisfactoriamente los objetivos iniciales. Ha sido un reto en el que nos hemos encontrado con problemas reales de la industria tecnológica, desde el diseño de la experiencia de usuario hasta las decisiones de infraestructura y despliegue en la nube. Nos sentimos muy satisfechos con la aplicación desarrollada ya que en el proceso hemos aprendido tantos aspectos técnicos como de organización de proyectos de software.

Mejoras Propuestas y Limitaciones

A pesar de contar con una aplicación sólida y funcional, una vez terminado el proyecto hemos identificado ciertas limitaciones de nuestra arquitectura y cosas que haríamos distintas si tuviéramos más tiempo y presupuesto.

La principal limitación de nuestro sistema es la infraestructura. Cómo utilizamos el plan gratuito de *Render* para el backend, el servidor entra en modo de hibernación si pasa 15 minutos sin recibir peticiones. Esto provoca un cold start (arranque en frío), el primer usuario que entra a la aplicación tras la hibernación puede notar que el login o la carga del mapa tardan unos 50 segundos. Esto es simplemente porque la máquina se está encendiendo. Una vez arrancado, el sistema funciona de forma completamente normal.

Por otra parte, el proyecto cuenta con otra limitación relacionada con la naturaleza de las publicaciones del BOE. A veces, en un mismo anuncio (mismo ID), no encontramos una sola subasta, sino un lote entero (Ej: dos plazas de garaje y un piso), todo redactado en el mismo párrafo. Para no complicar demasiado el esquema y debido a que con las subastas normales nos sobra para demostrar el funcionamiento de la app, decidimos que la IA se quede únicamente con la primera subasta del lote, descartando las demás. Para una primera versión no está mal, sin embargo, si se deseara lanzar como un producto competitivo no podríamos permitirnos ocultar a los usuarios potenciales buenas ofertas.

Si tuviéramos que volver a hacer el proyecto, haríamos varias cosas de otra forma, en parte para suplir estas limitaciones que tiene la app. En primer lugar diseñaríamos el modelo de datos de manera diferente desde el primer día. En lugar de tener un ID del BOE para cada subasta, estructuraríamos la base de datos para tener para cada anuncio N lotes. En el prompt de la IA, le diríamos que devolviera un array de objetos en vez de solo uno. Como nos dimos cuenta de esta necesidad cuando el proyecto ya estaba avanzado, refactorizar nos suponía un trabajo extra bastante alto para los sprints que quedaban.

Otra decisión arquitectónica que cambiaríamos sería la gestión de la cola de trabajo de la IA. Ahora utilizamos la base de datos como si fuera una cola (estado "PENDIENTE"), sin embargo, de cara a la escalabilidad esto es una mala práctica. Si lo volviéramos a montar integraríamos un sistema de colas real en memoria (como RabbitMQ). Esto nos daría un mayor control y evitaría saturar la base de datos.

Por último, podríamos plantear un proceso de extracción de datos más potente. En esta versión únicamente analizamos el texto XML de los anuncios, sin embargo, en varios de ellos hemos descubierto información adicional adjunta bastante valiosa (cargas, referencias

catastrales) en PDFs. Si rehiciéramos la ingesta, podríamos incluir una etapa de OCR para obtener la información de esos PDFs y mandarle a la IA mucho más contexto.